

Premiers pas avec Qt Quick-PyQt

Première partie : présentation de QML, Qt Quick et création d'interfaces simples

Par Charles-Élie Gentil  

Date de publication : 19 octobre 2013

Dernière mise à jour : 4 février 2015

TOUT PUBLIC

Cette première partie se veut être une première approche dans la programmation avec Qt Quick. Contrairement aux parties qui suivront, à ce stade aucune notion en Python n'est nécessaire. Nous y verrons ce que sont Qt Quick et QML et comment créer une interface graphique basique.

Commentez

En complément sur Developpez.com

- [Introduction à Qt Quick pour les développeurs C++](#)
- [Configurer Qt Creator pour Python](#)

I - Introduction.....	3
II - Présentation.....	3
II-A - Qt Quick.....	3
II-B - QML.....	4
III - Création d'une interface simple.....	4
III-A - Afficher un simple rectangle.....	4
III-B - Personnalisation avec d'autres éléments de base.....	6
IV - Conclusion.....	8
V - Remerciements.....	8

I - Introduction

Cet article a pour but de constituer une première approche sur l'utilisation des modules QML et Qt Quick avec PyQt 5. Il est donc nécessaire de s'assurer avant toute chose de l'installation de Python 3 et PyQt 5.

L'article complet est proposé en trois parties :

- la première présente QML, Qt Quick et la création d'interfaces simples (*article que vous allez lire ci-dessous*) ;
- la seconde s'occupe de la création d'interfaces graphiques plus développées et de l'interaction avec Python ;
- la troisième et dernière se préoccupera de la réalisation d'un programme complet avec Qt Quick et PyQt 5.

Chacune de ces parties permettra au lecteur d'acquérir des notions de plus en plus élaborées, le menant progressivement à créer un éditeur de texte.

Cette première partie présente dans un premier temps le langage QML ainsi que le module Qt Quick et leurs possibilités.

Dans un second temps, on apprendra à créer des interfaces simples afin d'appréhender la manière de coder en QML.

II - Présentation

Avant de coder votre première interface graphique avec ces nouveaux outils, je vous propose dans un premier temps une petite définition de ce que sont Qt Quick et QML.

II-A - Qt Quick

Qt Quick est à considérer comme un framework graphique regroupant le langage QML, les modules associés à ce langage (anciennement Qt Declarative) et Qt Creator. Ce framework graphique nouvelle génération fait partie intégrante de Qt.

Qt Quick permet au développeur de créer des interfaces personnalisables avec des rendus et transitions fluides plus facilement qu'avec les modules courants (Qt Widgets, Qt GUI...), notamment dans le but de tendre vers une interface proche de ce qui existe actuellement dans le domaine des applications pour tablette ou smartphone.

D'autre part, Qt Quick aidera les designers dans l'élaboration d'une UI qui sera partagée entre ces derniers et les développeurs. Prenons le cas d'un projet dans lequel la partie graphique et la partie application sont gérées par deux équipes distinctes. Avec les outils actuels, les designers sont dépendants des développeurs, puisqu'ils ne peuvent pas tester l'interface et ses effets visuels sans passer par du code « application ».

La grande nouveauté avec Qt Quick est qu'il est possible de tester le rendu de l'interface sans pour autant dépendre de la partie traitement de données. Ceci permet au sein d'un projet de développement le travail collaboratif entre la partie graphique et la partie application en toute indépendance.

Pour les développeurs Python, l'intérêt d'utiliser Qt Quick est encore plus grand. Dans le cas d'une interface créée avec Qt Designer, le développeur Python doit dans la plupart des cas :

- créer l'interface graphique sous Qt Designer ;
- convertir le ou les .ui en .py.

Avec Qt Quick, l'étape 2 est tout simplement supprimée, les changements d'interface seront pris en compte directement.

II-B - QML

QML est au cœur de Qt Quick. Il s'agit là d'un langage déclaratif, comprenez par là « je ne veux pas savoir comment tu fais, je veux que tu le fasses ». Dit comme ça, cela peut être offusquant, mais en réalité ce paradigme apporte aux développeurs et aux designers en particulier une grande souplesse.

Prenons l'exemple d'un simple bouton. Aujourd'hui, vous pouvez utiliser le widget `QPushButton` pour faire apparaître un bouton, mais vous allez rapidement être limités à des paramètres simples : pour atteindre le rendu attendu, il faudra modifier le widget plus en profondeur.

C'est là que QML devient intéressant, car il permet très simplement d'obtenir la mise en forme voulue par l'équipe sans être obligé d'y passer des heures. Ceci paraît sûrement abstrait dans l'état actuel des choses, mais vous verrez plus tard comment tout ceci se présente... À ce stade de l'apprentissage, il faut bien garder une part de mystère.

D'autre part, QML est un langage interprété permettant ainsi au designer de tester son travail sans passer par la phase de compilation. Sa productivité n'en sera alors qu'améliorée.

Pour finir, sachez qu'un fichier QML est un fichier avec l'extension `.qml`. Cela paraît un peu évident, mais il est toujours bon de le rappeler. Un fichier avec l'extension `.qml` est appelé un document QML.

La syntaxe d'un tel document est assez proche de celle d'un fichier CSS. En effet, un fichier QML est composé d'un ou plusieurs éléments auxquels on attribue des propriétés, chaque élément pouvant par la suite être réutilisé dans d'autres documents QML.

Contrairement à CSS, chaque élément peut aussi avoir des éléments enfants. Dans ce cas l'élément parent est à considérer comme un conteneur et les éléments enfants le contenu.

III - Création d'une interface simple

Après avoir dégrossi Qt Quick et QML, cette section commence le code en QML.

III-A - Afficher un simple rectangle

Afin d'accéder dans votre document aux éléments QML fournis dans Qt Quick, celui-ci devra toujours commencer par l'import d'un ou plusieurs modules. Pour commencer, nous travaillerons dans les exemples ci-dessous avec l'import de Qt Quick.

Revenons sur nos imports. Les imports en QML pourraient être assimilés à ceux que nous avons l'habitude de faire dans nos scripts Python :

Import en Python

```
import PyQt5
```

En QML, un import se fait en précisant le module que l'on souhaite importer ainsi que sa version séparée par un espace du nom. Ici, cela donnerait :

Import en QML

```
import QtQuick 2.0
```

Créons maintenant notre premier document QML. Ouvrez votre éditeur de texte préféré, voire Qt Creator si vous l'avez installé et créez un nouveau fichier vierge.

Si vous travaillez avec Qt Creator, notamment pour le code QML afin de bénéficier d'aides comme la coloration syntaxique par exemple, il est préférable de choisir **Fichiers et classes > Général > Fichier Texte** afin de ne pas griller les étapes. En effet, si vous choisissez l'option **Projet > Applications > Qt Quick 2 UI**, Qt Creator créera un nouveau fichier pré-rempli, ce que nous ne voulons pas actuellement.

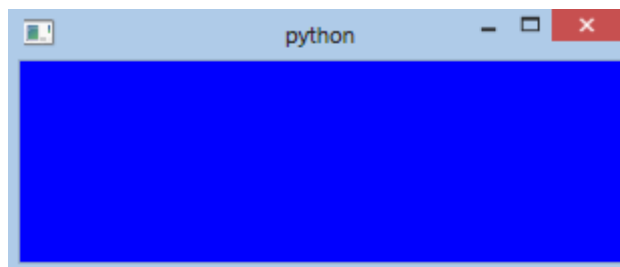
Vous pouvez maintenant enregistrer ce document sous le nom MyRectangle.qml.

 *Il est obligatoire de commencer le nom de votre document par une majuscule. J'en expliquerai plus tard la raison.*

Rappelez-vous, QML est un langage déclaratif. C'est-à-dire que l'on écrit ce que l'on veut obtenir. Dans l'exemple qui suit, je vous propose de créer un simple rectangle de largeur 300 pixels, de hauteur 100 pixels et de couleur bleue. Voici le code nécessaire :

```
MyRectangle.qml
import QtQuick 2.0
Rectangle {
    width: 300
    height: 100
    color: "blue"
}
```

Voilà, vous venez de créer votre première interface graphique en QML. Voici le résultat obtenu :



Vous voulez certainement voir le résultat de vos propres yeux. Avant de vous expliquer comment faire, je vous propose de commenter le code ci-dessus.

Comme on l'a vu plus haut, un document QML doit commencer par un import permettant d'accéder à la bibliothèque des éléments QML voulue. Voici donc comment se fait un import en QML :

```
MyRectangle.qml
import QtQuick 2.0
```

Nous souhaitons ensuite créer un rectangle. Le rectangle fait partie des éléments de base dans Qt Quick et il s'appelle ainsi :

```
MyRectangle.qml
import QtQuick 2.0
Rectangle {
}
```

Notez la syntaxe, un élément doit toujours :

- avoir son nom qui commence par une majuscule ;
- avoir après son nom une accolade ouverte qui délimite le début des propriétés.

Une fois notre rectangle appelé, il va falloir lui donner les caractéristiques voulues. Pour rappel, nous souhaitons que notre rectangle ait :

- une largeur de 300 pixels ;
- une hauteur de 100 pixels.

Nous allons appliquer à l'élément Rectangle les propriétés **width**, **height** et **color** avec les valeurs souhaitées.

Notez que **width** et **height** attendent une valeur de type `int`, tandis que **color** attend une valeur de type `string`. Si vous ne respectez pas ceci, une jolie erreur se produira et votre interface ne sera pas affichée.

```
MyRectangle.qml
import QtQuick 2.0
Rectangle {
    height: 100
    width: 300
    color: 'blue'
}
```

Comment visionner le résultat ? Deux possibilités :

- soit en utilisant `qmlscene.exe` intégré à PyQt 5 ;
- soit en créant un code Python affichant le document QML.

Dans un premier temps nous utiliserons `qmlscene.exe`. N'oublions pas que l'avantage de Qt Quick et QML est de pouvoir tester nos UI sans avoir à coder autre chose.

Sous Windows, par défaut pour Python 3.3 et PyQt 5, `qmlscene.exe` se trouve dans le répertoire `C:\Python33\Lib\site-packages\PyQt5`. Pour vous simplifier la vie vous pouvez mettre ce chemin dans votre path si ce n'est pas encore le cas. Attention cependant à ne pas mélanger plusieurs versions de PyQt ou Qt, `qmlscene` étant aussi disponible avec ce dernier.

Sous Linux, en théorie tout est déjà configuré correctement.

Placez-vous dans le répertoire où est enregistré votre document `MyRectangle.qml` et lancez la commande :

```
qmlscene MyRectangle.qml
```

Notez que vous pourriez aussi lancer `qmlscene` n'importe où :

```
qmlscene
```

Une boîte de dialogue va alors apparaître afin que vous puissiez aller chercher le document QML voulu.

Et voici votre rectangle qui apparaît.

III-B - Personnalisation avec d'autres éléments de base

QML permet de créer et de tester simplement et rapidement une première interface graphique. Cependant, l'exemple est loin d'être très développé. Je vous propose donc maintenant de personnaliser un peu plus le rectangle en y positionnant un texte et une image.

Qt Quick offre dans les éléments de base, au même titre que `Rectangle`, les éléments `Text` et `Image`. Nous voulons que notre élément `Rectangle` soit un conteneur et les éléments `Text` et `Image` des contenus : `Rectangle` est donc parent de `Text` et `Image`.

Voici comment les ajouter à notre projet :

MyRectangle.qml

```
import QtQuick 2.0
Rectangle {
    width: 300
    height: 100
    color: "blue"
    Text {
    }
    Image {
    }
}
```

Il ne reste plus qu'à ajouter les propriétés voulues pour afficher du texte et une image. Voici le code que je vous propose dans l'immédiat :

Exemple.qml

```
import QtQuick 2.0
Rectangle {
    width: 300
    height: 100
    color: "blue"
    Image {
        source: "http://ceg.developpez.com/images/dvp.png"
    }
    Text {
        text: "Qt Quick est formidable"
    }
}
```

Commentons ces quelques lignes complémentaires.

L'élément Image attend notamment la source de l'image à afficher dans la propriété **source**. Pour l'exemple, j'ai choisi une image sur un serveur distant, mais j'aurais pu aussi choisir une image présente en local (ce qui sera fait dans la prochaine partie de l'article).

L'élément Text attend, quant à lui, un texte via la propriété **text**.

Essayez votre interface : vous remarquerez que la mise en page est loin d'être très chouette. Pour l'améliorer, ces éléments disposent d'autres propriétés pour les positionner aisément dans notre fenêtre. Notamment, les propriétés de type **anchors** permettent d'ancrer les éléments.

Exemple.qml

```
import QtQuick 2.0

Rectangle {
    width: 300
    height: 100
    color: "grey"
    Image {
        source: "http://ceg.developpez.com/images/dvp.png"
        anchors {
            bottom: parent.bottom
            topMargin: 30
            horizontalCenter: parent.horizontalCenter
        }
    }
    Text {
        text: "Qt Quick c'est formidable"
        styleColor: "#ffffff" //une autre façon de déclarer la couleur de l'item.
        anchors {
            top: parent.top; topMargin: 10 ; horizontalCenter: parent.horizontalCenter
        }
    }
}
```

Testez votre application, normalement vous devriez obtenir ceci :



Ce qui a été ajouté est assez explicite et ne nécessite pas plus de commentaires. Cependant, j'aimerais attirer votre attention sur le fait que vous pouvez définir plusieurs propriétés sur une même ligne. Il faudra uniquement veiller à bien séparer chaque propriété par un point-virgule.

IV - Conclusion

Et voilà, au terme de cette première partie, vous devez être en mesure de créer vos premières interfaces avec Qt Quick.

Dans les parties qui suivront nous verrons comment créer des applications plus complexes (partie 2), puis nous verrons comment interagir entre du code QML et du code Python.

V - Remerciements

Je remercie tout particulièrement **Thibaut Cuvelier** et **Fabien (f-leb)** pour leurs précieux conseils et leurs relectures attentives.